

Reviewer Pack — Edge-Expansion Rank Bound on Read-Twice BPs for Tseitin

Entry 22abf1f27d7f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 20:26:29 UTC

Edge-Expansion Rank Bound on Read-Twice BPs for Tseitin

Entry ID: 22abf1f27d7f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 20:26:29 UTC

1. Conjecture

Field A (mathematical branch): Discrepancy / sign-rank theory of communication matrices (real-rank as a discrepancy proxy, in the spirit of Razborov-Sherstov) **Field B** (complexity object): Syntactic read-twice branching program size for Tseitin formulas on 3-regular expanders

Statement:

Let G be a connected 3-regular graph on m vertices with edge expansion $h(G) := \min_{1 \leq |S| \leq m/2} |\partial S|/|S|$, and let $\sigma: V(G) \rightarrow \{0, 1\}$ be any odd charging. Let π be the median edge bipartition (first $\lfloor |E|/2 \rfloor$ edges by canonical lex order vs. the rest), and let $M_{\{G, \sigma, \pi\}} \in \{0, 1\}^{\lfloor |E|/2 \rfloor \times 2^{\lfloor |E|/2 \rfloor}}$ be the bipartite truth-table of the Tseitin function $f_{\{G, \sigma\}}$. Conjecture (BP-READTWICE-Tseitin Rank): $\log_2 \text{rank}_{\mathbb{R}}(M_{\{G, \sigma, \pi\}}) \geq (1/4) \cdot h(G) \cdot m$ for every such (G, σ) ; equivalently, since every syntactic read-twice BP P satisfies $\text{rank}_{\mathbb{R}}(M_{\{f, \pi\}}) \leq \text{size}(P)^2$, every read-twice BP for $f_{\{G, \sigma\}}$ has size $\geq 2^{\{h(G) \cdot m/8\}}$.

Rationale (proposer's reasoning):

Read-once BPs admit $\text{rank}(M_f) \leq \text{size}(P)$ under matching orderings, and the only known size lower bounds for Tseitin on expanders go through this rank/width route; pushing the exponent from 1 to 2 captures exactly the 'one extra read' of read-twice and lets edge-expansion (the same combinatorial gadget Ben-Sasson-Wigderson used for resolution width) drive a BP size bound. Real-rank of the communication matrix is a clean, purely combinatorial discrepancy proxy, sidestepping algebrization (the bound uses a non-algebraic graph invariant $h(G)$) and natural-proofs (the property is not P-time on truth tables, only on BPs). If true it would yield the first quantitative read-twice lower bound for an explicit family beyond IP₂.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 481f8aa7188718cc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For $m \in \{4, 6, 8, 10, 12, 14\}$, generate 30 random connected 3-regular graphs with random odd σ (180 instances total). Compute $\log_2 \text{rank}_{\mathbb{R}}(M_{\{G, \sigma, n\}})$ and $h(G)$ by brute force. Support requires $\log_2 \text{rank} \geq 0.25 \cdot h(G) \cdot m$ on every instance, AND OLS regression of $\log_2 \text{rank}$ on $h(G) \cdot m$ yields slope ≥ 0.25 with $p < 0.01$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formulas read-twice branching program lower bound expander - communication matrix rank Tseitin expander graph lower bound - syntactic read-k branching program size Tseitin edge expansion

Top relevant hits considered: - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1210.1461v1>] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound - [<http://arxiv.org/abs/1411.3958v3>] Edge-state enhanced transport in a 2-dimensional quantum walk - [<http://arxiv.org/abs/1301.1425v2>] Pebbling, Entropy and Branching Program Size Lower Bounds

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3_regular_graph(m):
        while True:
            vertices = list(range(m))
            edges = []
            for v in vertices:
                neighbors = random.sample(vertices, 2)
                if v not in neighbors and (v, neighbors[0]) not in edges and (v, neighbors[1]) not in edges:
                    edges.append((v, neighbors[0]))
```

```

        edges.append((v, neighbors[1]))
    if len(edges) == 3 * m // 2:
        return vertices, edges

def compute_edge_expansion(vertices, edges):
    n = len(vertices)
    min_cut_size = float('inf')
    for i in range(1, n // 2 + 1):
        for S in itertools.combinations(vertices, i):
            cut_size = sum(1 for u, v in edges if (u in S and v not in S) or (v in S and u not in S))
            min_cut_size = min(min_cut_size, cut_size)
    return min_cut_size / n

def compute_median_edge_bipartition(edges):
    sorted_edges = sorted(edges)
    mid = len(sorted_edges) // 2
    first_half = sorted_edges[:mid]
    second_half = sorted_edges[mid:]
    return (first_half, second_half)

def compute_truth_table(vertices, edges, sigma):
    n = len(vertices)
    m = len(edges)
    truth_table = [[0] * (1 << m) for _ in range(1 << n)]
    for i in range(1 << n):
        for j in range(1 << m):
            value = 1
            for v in vertices:
                if sigma[v] == 1 - ((i >> v) & 1):
                    value *= (-1)
            truth_table[i][j] = value
    return truth_table

def compute_rank(matrix):
    rank = 0
    m, n = len(matrix), len(matrix[0])
    for col in range(n):
        pivot_row = -1
        for row in range(m):
            if matrix[row][col] != 0:
                if pivot_row == -1:
                    pivot_row = row
                else:
                    factor = matrix[pivot_row][col] / matrix[row][col]
                    for c in range(n):
                        matrix[row][c] -= factor * matrix[pivot_row][c]
        rank = sum(1 for row in range(m) if any(matrix[row]))
    return rank

n_values = [4, 6, 8, 10, 12, 14]
results = []

for n in n_values:
    m = 3 * n // 2
    for _ in range(5): # Ensure we test at least 30 instances per seed
        vertices, edges = generate_3_regular_graph(m)

```

```

sigma = {v: random.choice([0, 1]) for v in vertices}
h_G = compute_edge_expansion(vertices, edges)
first_half, second_half = compute_median_edge_bipartition(edges)
truth_table = compute_truth_table(vertices, first_half + second_half, sigma)
rank = compute_rank(truth_table)

results.append({
    "metric_name": "log2_rank",
    "metric_value": math.log2(rank),
    "instances_tested": 1,
    "conjecture_holds": math.log2(rank) >= 0.25 * h_G * m,
    "counterexample": "" if math.log2(rank) >= 0.25 * h_G * m else f"m={m}, rank={rank}"
})

return {
    "seed": seed,
    "metric_name": "log2_rank",
    "metric_value": sum(result["metric_value"] for result in results) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": all(result["conjecture_holds"] for result in results),
    "counterexample": next((result["counterexample"] for result in results if not result["conjecture_holds"]), None)
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode 124) without producing any RESULT line or instance data, so neither the support nor falsification conditions can be evaluated. | next: Reduce the instance set (e.g., $m \in \{4,6,8\}$ with 10 graphs each) or optimize the rank/expansion computation (sparse linear algebra, cached BFS) to fit within the time budget, then rerun.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	165045
2	preregistration	claude_max	opus	0	0	5697
3	novelty	claude_max	opus	0	0	5128
4	novelty	claude_max	opus	0	0	14141
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	84035
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	127328
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	255771
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15311
9	judge	claude_max	opus	0	0	9772

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 682229 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/22abf1f27d7f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/22abf1f27d7f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/22abf1f27d7f.tar.gz` (if generated)